

Data bases 2

Written exam 1/9/2022: JPA exercise

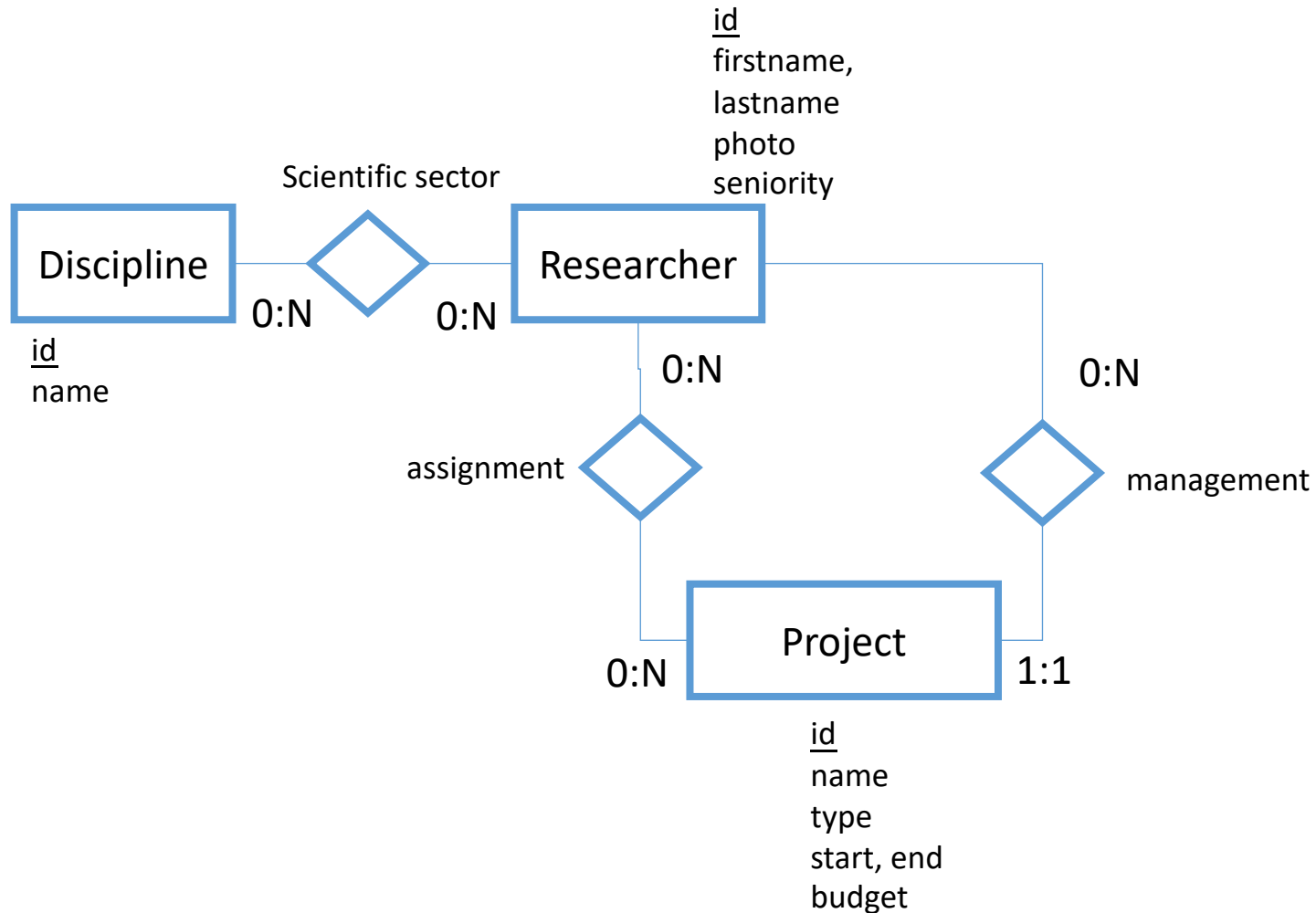
Specifications

- An application lets a researcher edit the projects s/he is responsible for. A project has a name, a start and end date a type and a budget. A researcher can create projects and assign other researchers to them. Researchers have a name, an id, a photo, and a seniority level. Researchers also pertain to one or more disciplines (e.g., computer science, automation, telecommunications, etc.). The application lets a researcher access the list of projects s/he manages (in descending order of end date) and the list of projects s/he works in. By selecting one project s/he can see the details of the project including the list of the researchers that work in it / manage it with the disciplines they belong to. Researchers are in the order of hundreds, projects are in the order of thousands and disciplines in the order of tens.

Questions

- Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application.
- Comment the design choices you have made.

Entity Relationship



Relational model

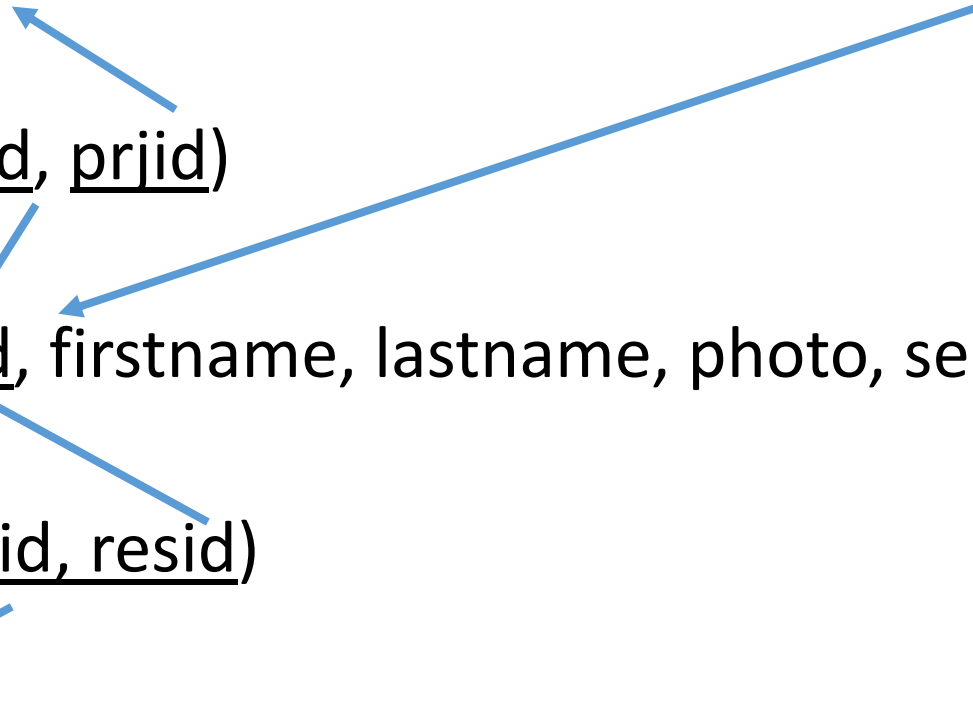
project(id, name, start, end, type, budget, manager)

res_prj(respid, prjid)

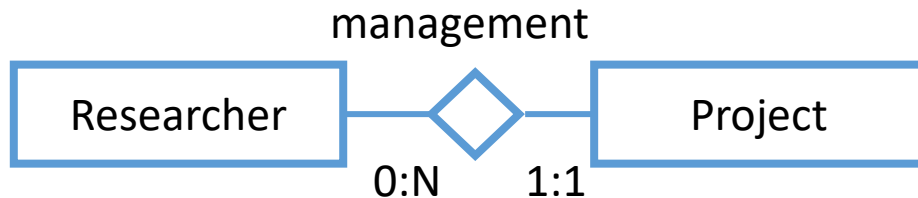
researcher(id, firstname, lastname, photo, seniority)

disc_res(discid, resid)

discipline(id, name)

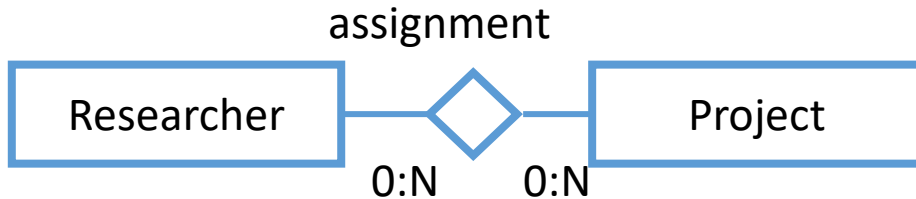


Relationship “management”



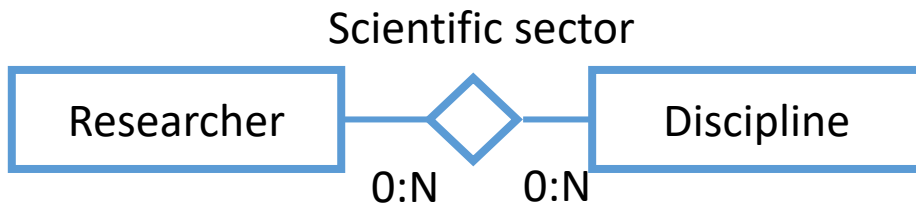
- Researcher → Project
@OneToMany is necessary to get the projects managed by the logged in user
 - @OrderBy("end DESC")
 - Owner = project
 - FetchType.Eager
- Project → Researcher
@ManyToOne is requested to show the researcher that manages it

Relationship “assignment”



- Project → Researcher @ManyToMany is necessary to show the researchers that work in a project
 - Owner = either Researcher or Project
 - FetchType.EAGER to let the client access the researchers working in a project via relationship navigation
- Researcher → Project @ManyToMany is requested to show the list of projects s/he works in.
 - FetchType.EAGER

Relationship “scientific sector”



- Researcher → discipline
@ManyToMany is necessary to show the disciplines of a researcher
 - Owner
 - Either Researcher or Discipline
 - FetchType.EAGER to let the client access the disciplines of a researcher via relationship navigation
- Discipline → Researcher
@ManyToMany not requested, but mapped for simplicity

Entity Researcher

```
@Entity
public class Researcher implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String firstname;
    private String lastname;
    @Basic(fetch = FetchType.LAZY)
    @Lob
    private byte[] photo;
    private String seniority;

    @ManyToMany(mappedBy = "researchers", fetch =
FetchType.EAGER)
    private List<Project> assignedProjects;

    @OneToMany(mappedBy = "manager", fetch =
FetchType.EAGER)
    @OrderBy("end DESC")
    private List<Project> managedProjects;

    @ManyToMany(mappedBy = "researchers", fetch =
FetchType.EAGER)
    private List<Discipline> disciplines;
```

Entity Project

```
@Entity
@NamedQuery(name="Project.findAll", query="SELECT p FROM Project p")
public class Project implements Serializable {
    ...

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int id;
    private String name;
    private String type;
    private int budget;

    @Temporal(TemporalType.DATE)
    private Date start;
    @Temporal(TemporalType.DATE)
    private Date end;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name="res_prj",
        joinColumns={@JoinColumn(name="projid")},
        inverseJoinColumns={@JoinColumn(name="resid")}
    )
    private List<Researcher> researchers; //owner

    @ManyToOne
    @JoinColumn(name="head")
    private Researcher manager; //owner
```

Entity Discipline

```
@Entity
public class Discipline implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int id;
    private String label;

    @ManyToMany
    @JoinTable(
        name="dis_res",
        joinColumns={@JoinColumn(name="discid")},
        inverseJoinColumns={@JoinColumn(name="resid")}
    )
    private List<Researcher> researchers; //owner
```